# AI assist (optional)

OpenTrack can use an AI model to help with issues. It's **off by default** and only ever runs when you turn it on and point it at a provider. Whatever it does is only ever a suggestion or a shortcut — a person always stays in control.

## What the AI can do

- **Smart triage** — on the **New issue** page, a ■ **Suggest with AI** button

reads your summary/details and fills in a severity, priority, category, and proposed tags. You accept or change them before saving.

- **Plain-English search** — on the **Issues** list, an **■ Ask in plain

English** box turns a request like *"high-priority crashes nobody has touched in a month"* into the normal filters (status, severity, priority, keywords, stale, project). It only ever produces a filter you could have set by hand, and it can only match projects you're allowed to see — so it never widens your access.

You are **not** locked into one AI company. OpenTrack talks to two kinds of provider, so you can pick what fits your budget and privacy needs.

## Which AI can I use?

| You want… | Provider setting | Notes |
|---|---|---|
| **Anthropic Claude** (what the author uses) | `anthropic` | Cloud. Needs an Anthropic API key. |
| **OpenAI** (ChatGPT models, e.g. GPT-4o mini) | `openai` | Cloud. Needs an OpenAI API key. |
| **Azure OpenAI** | `openai` + `BaseUrl` | Cloud, your Azure resource. |
| **Groq / OpenRouter / other hosted** | `openai` + `BaseUrl` | Cloud. Any OpenAI-compatible endpoint. |
| **Ollama** (free, runs on your own PC) | `openai` + `BaseUrl` | **Local — no key, no data leaves your machine.** |
| **LM Studio** (free, runs on your own PC) | `openai` + `BaseUrl` | **Local — same privacy benefit.** |

The `openai` setting means "any service that speaks OpenAI's Chat Completions format." A huge number of tools — including the free local ones — do, which is why one setting covers so many options.

## Important: who pays?

The cloud providers bill an **API account** at that provider — this is **separate** from any monthly chat subscription. In particular, an Anthropic API key is billed to your **Anthropic API account**, which is **not** the same as a Claude Pro/Max subscription; turning on AI here does **not** draw from a Claude.ai plan. The **local** options (Ollama, LM Studio) are **free** and run on your own computer — no account, no per-use charge.

Cloud costs for these features are small — a triage suggestion is typically a fraction of a cent — but always check the provider's current pricing, and set a spend limit where the provider offers one.

---

## Step-by-step: get an Anthropic (Claude) API key

1 Open a browser and go to **<https://console.anthropic.com>**. This is the

**Anthropic Console** (the developer/API site) — *not* claude.ai, the chat site. They are separate accounts even if you use the same email.

1 **Sign in**, or click **Sign up** and create an account (email + password, or

"Continue with Google"). Verify your email if asked.

1 Once you're in the Console, look at the **left sidebar** (or the gear/■

**Settings** menu) and click **API Keys**. (Direct link: <https://console.anthropic.com/settings/keys>.)

1 Click **Create Key** (sometimes labeled "Create API Key").

2 Give it a name you'll recognize later, like `OpenTrack`, and click **Create**

/ **Add**.

1 The key is shown **once**. It looks like `sk-ant-api03-XXXXXXXX…`. Click

**Copy** and paste it somewhere safe *right now* — you can't view it again later (only delete it and make a new one).

1 **Add credit so the key works.** A brand-new API account usually has a $0

balance and calls will fail until you fund it. In the sidebar go to **Billing** → **Plans** / **Buy credits**, add a payment method, and buy a small amount (even $5 is plenty for triage). While you're there, set a **monthly spend limit** so it can never surprise you.

1 Keep that `sk-ant-…` key handy for the "Turn it on" step below.

## Step-by-step: get an OpenAI API key

1 Go to **<https://platform.openai.com>** — the **OpenAI developer platform**,

*not* chatgpt.com. Different account from a ChatGPT Plus subscription.

1 **Sign in** or **Sign up** and verify your email (and phone, if asked).

2. Click your **profile icon (top-right)** → **View API keys**, or go straight to

<https://platform.openai.com/api-keys>.

1. Click **Create new secret key**, name it `OpenTrack`, and click **Create**.
2. Copy the key (it starts with `sk-…`) **now** — it's shown only once.
3. Go to **Settings** → **Billing**, add a payment method, and add a little credit.

Set a **usage limit** while you're there.

1. Pick a cheap, capable model name for the config below — `gpt-4o-mini` is a

good default.

## Free & private option: run it locally with Ollama (no key at all)

If you'd rather **not** send issue text to any cloud, run a model on your own machine. Nothing leaves your computer.

1. Download and install **Ollama** from **<https://ollama.com>** (Windows, Mac,

Linux).

1. Open a terminal and pull a model, e.g.:

```
ollama pull llama3.1
```

Ollama then serves an OpenAI-compatible API at `http://localhost:11434/v1`.

1. Use the "Local (Ollama)" settings below — **no API key needed**.

*(LM Studio works the same way; it serves on `http://localhost:1234/v1`.)*

### *Run the local model on another machine (e.g. a Raspberry Pi) on your network*

The local engine does **not** have to run on the same computer as OpenTrack. If you have Ollama (or LM Studio) running on another box on your LAN — a Raspberry Pi, a NAS, a spare desktop — just point `BaseUrl` at **that machine's IP** instead of `localhost`:

```
OpenTrack:Ai:BaseUrl = http://192.168.1.50:11434/v1     (the Pi's LAN IP)
```

Two things to set up on the Pi so it's reachable from other machines:

1. **Let Ollama listen on the network, not just itself.** By default it only

answers `localhost`. Start it with the environment variable `OLLAMA_HOST=0.0.0.0:11434` so it accepts LAN connections. (On Raspberry Pi OS / Linux you can add `Environment=OLLAMA_HOST=0.0.0.0:11434` to the ollama systemd service, then `sudo systemctl restart ollama`.)

1. **Allow port 11434** through the Pi's firewall if it has one enabled.

Then OpenTrack (on your Beelink, laptop, wherever) calls the Pi across the LAN and the issue text still never leaves your own network.

**Reality check on speed:** a Raspberry Pi has no GPU, so it can only run small models and will be **noticeably slower** than a cloud call — fine for the occasional triage/summary, less so for rapid-fire use. A Pi 5 with 8 GB can run a small model (roughly 1–3 billion parameters, e.g. `llama3.2:3b`); stick to small models and keep expectations modest. This is why the OpenAI provider is given a longer (60-second) timeout than the cloud path.

## Good hardware for local AI (and what won't help)

You don't need much for OpenTrack's use — triage and summaries are short, bursty calls, not constant heavy use. Two things decide whether a machine is "good" enough: **enough memory to hold a decent model**, and (nice-to-have) **some GPU/NPU acceleration** for speed. Of the two, memory matters most.

**Memory is the main number — 16 GB is the sweet spot, 24–32 GB gives headroom:**

| RAM | What it can run | Verdict |
|---|---|---|
| 8 GB | ~3B models only | Works, but weaker suggestions |
| 16 GB | 7–8B models (~5–6 GB) | The practical sweet spot for triage/summaries |
| 24–32 GB | 13–14B models, or the AI **and** OpenTrack on one box | Comfortable headroom |
| 64 GB+ | 30B+ models | Overkill for this |

**What actually helps speed:**

- **Apple Silicon (Mac mini M-series)** — its unified memory doubles as fast GPU

memory and Ollama uses it out of the box, so even a small Mac mini feels snappy where a CPU-only box merely works. The easiest strong option, and a quiet, low-power always-on box. **Which one:**

- **New: a current M4 Mac mini** — fastest and most efficient, and its base

model now includes 16 GB. ~$599 (16 GB) or ~$799 (24 GB, the comfortable pick). The pricier M4 **Pro** has much higher memory bandwidth (faster generation) but is overkill for occasional triage/summaries.

- **Budget: a used/refurbished M1 (2020) or M2 (2023) mini with 16 GB** — both

run a 7–8B model well.

- **Avoid any Intel Mac mini** (2018 and earlier): no Apple Silicon, so none of

the unified-memory/Metal advantage — it's just a CPU x86 box.

- Buy enough RAM up front: on Apple Silicon the memory is soldered and can't be

upgraded later, and RAM doubles as the model's working memory.

- **An NVIDIA GPU** (even a modest one) — CUDA acceleration that Ollama/llama.cpp

love. Fastest, but pricier and more setup.

- **A CPU-only mini PC** (e.g. an Intel/AMD box like a Beelink) — runs local

models on the CPU at a few tokens/second. Not instant, but perfectly fine for occasional triage/summaries, and often a machine you **already own**. If it's the same box already running OpenTrack, just install Ollama alongside and point `BaseUrl` at `localhost` — no second machine, no network setup. A modern mini PC with **16–24 GB** is a sensible home for local AI here.

**What won't help (for text AI):**

- **Vision NPUs / "AI HAT" add-on boards** (e.g. a Raspberry Pi AI HAT+ with a

Hailo accelerator). These are built for **computer-vision** models; Ollama and llama.cpp can't offload language models onto them, so they sit idle for OpenTrack's purposes. Great for camera projects — not for this. Put the money toward **RAM** or **Apple Silicon** instead.

- **Weak integrated graphics** (e.g. Intel UHD): don't count on iGPU acceleration;

the model will just run on the CPU. That's okay for occasional use.

**Bottom line:** for personal, occasional use, a mini PC with **16–24 GB of RAM** is plenty — ideally one you already have. Only reach for Apple Silicon or an NVIDIA GPU if you want it to feel snappy rather than merely work.

---

## Turn it on

Put the settings in configuration — `appsettings.json`, environment variables, or .NET user-secrets. **Prefer environment variables or user-secrets for the key so it isn't committed to source control.** As environment variables, replace each `:` with `__` (double underscore), e.g. `OpenTrack__Ai__Enabled=true`.

**Claude (Anthropic):**

```
OpenTrack:Ai:Enabled  = true
OpenTrack:Ai:Provider = anthropic
OpenTrack:Ai:ApiKey   = sk-ant-...            (your Anthropic key)
OpenTrack:Ai:Model    = claude-haiku-4-5-20251001   (fast & inexpensive)
```

**OpenAI (cloud):**

```
OpenTrack:Ai:Enabled  = true
OpenTrack:Ai:Provider = openai
OpenTrack:Ai:ApiKey   = sk-...                (your OpenAI key)
OpenTrack:Ai:Model    = gpt-4o-mini
```

**Local, free & private (Ollama):**

```
OpenTrack:Ai:Enabled  = true
```

```
OpenTrack:Ai:Provider = openai
OpenTrack:Ai:BaseUrl  = http://localhost:11434/v1     (or a LAN IP like
                                                       http://192.168.1.50:11434/v1
                                                       for a Pi/NAS/other machine)
OpenTrack:Ai:Model    = llama3.1
                                                (no ApiKey needed for local)
```

**Azure OpenAI / Groq / OpenRouter / other hosted:** use `Provider = openai`, set `BaseUrl` to that service's endpoint, `ApiKey` to its key, and `Model` to a model it offers.

Restart OpenTrack. On the **New issue** page you'll now see a ■ **Suggest with AI** button that fills in severity/priority/category and proposes tags.

## Privacy & safety

- **Opt-in.** With AI off (the default) or unconfigured, OpenTrack behaves exactly

as before and makes no AI calls at all.

- **Cloud providers see the text.** With a cloud provider on, a suggestion sends

that issue's summary/description to that provider. Don't enable a cloud provider for projects whose contents can't leave your environment — use a **local** engine (Ollama/LM Studio) instead, which keeps everything on your machine.

- **Server-side only.** The key is read from server configuration and is never

sent to the browser or stored in the database.

- **Best-effort.** Every AI result is a suggestion with a human in the loop; if a

call fails, issue creation is completely unaffected.